

EC400: Dynamic Programming

DMITRY MUKHIN
d.mukhin@lse.ac.uk

Introduction

- **Recursive approach** is at the heart of modern macro:
 - allows to get **global solution**
 - can accommodate **discrete choice**
 - compact way to define problem/equilibrium

Introduction

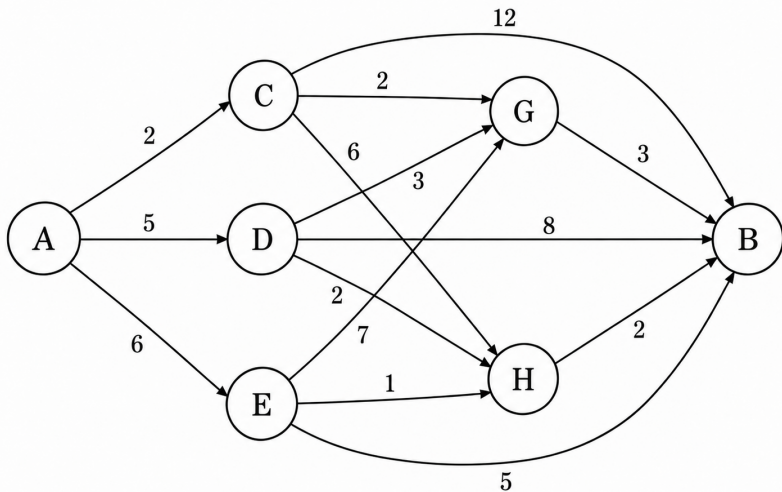
- **Recursive approach** is at the heart of modern macro:
 - allows to get **global solution**
 - can accommodate **discrete choice**
 - compact way to define problem/equilibrium
- Main limitation is the “**curse of dimensionality**”
 - problems with a large number of state variables

Introduction

- **Recursive approach** is at the heart of modern macro:
 - allows to get **global solution**
 - can accommodate **discrete choice**
 - compact way to define problem/equilibrium
- Main limitation is the “**curse of dimensionality**”
 - problems with a large number of state variables
- Key idea of recursive methods:
 - large number of periods and states complicate the analysis
 - but they are naturally ordered by time
 - convert large-scale system into a sequence of simple problems

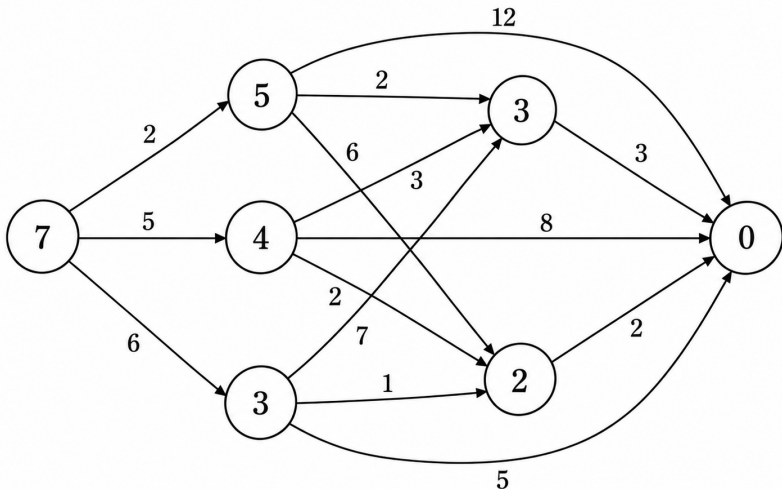
Example: Transportation Problem

Transportation network with multiple possible routes



Example: Transportation Problem

Dynamic programming values on the transportation network



PIE EATING PROBLEM

Two Methods

- Assume $u(C) = \log C$, $Y_t = 0$, $R = 1$, $T < \infty$

Two Methods

- Assume $u(C) = \log C$, $Y_t = 0$, $R = 1$, $T < \infty$
- The pie eating problem:

$$\max_{\{C_t, B_{t+1}\}} \sum_{t=0}^T \beta^t \log C_t$$

$$\text{s.t. } B_{t+1} = B_t - C_t, \quad B_{T+1} \geq 0$$

Two Methods

- Assume $u(C) = \log C$, $Y_t = 0$, $R = 1$, $T < \infty$
- The pie eating problem:

$$\max_{\{C_t, B_{t+1}\}} \sum_{t=0}^T \beta^t \log C_t$$

$$\text{s.t. } B_{t+1} = B_t - C_t, \quad B_{T+1} \geq 0$$

- **Sequential approach:** solves the system of (EE), (BC), (TVC)
 - $2 \times (T + 1)$ equations with $2 \times (T + 1)$ unknowns
 - computation burden increases as $T \rightarrow \infty$

Two Methods

- Assume $u(C) = \log C$, $Y_t = 0$, $R = 1$, $T < \infty$
- The pie eating problem:

$$\max_{\{C_t, B_{t+1}\}} \sum_{t=0}^T \beta^t \log C_t$$

$$\text{s.t. } B_{t+1} = B_t - C_t, \quad B_{T+1} \geq 0$$

- **Sequential approach**: solves the system of (EE), (BC), (TVC)
 - $2 \times (T + 1)$ equations with $2 \times (T + 1)$ unknowns
 - computation burden increases as $T \rightarrow \infty$
- **Recursive approach**: solves by backward induction
 - solve for C_t, B_{t+1} for each period starting from $t = T$
 - each step remains the same as $T \rightarrow \infty$

Backward Induction

① Period T :

$$\max_{B_{T+1} \geq 0} \log(B_T - B_{T+1})$$

Backward Induction

① Period T :

$$\max_{B_{T+1} \geq 0} \log(B_T - B_{T+1})$$

$$\Rightarrow B_{T+1} = 0, \quad C_T = B_T, \quad V_T = \log(B_T)$$

Backward Induction

① Period T :

$$\max_{B_{T+1} \geq 0} \log(B_T - B_{T+1})$$

$$\Rightarrow B_{T+1} = 0, \quad C_T = B_T, \quad V_T = \log(B_T)$$

② Period $T - 1$:

$$\max_{B_T} \left\{ \log(B_{T-1} - B_T) + \beta V_T(B_T) \right\}$$

Backward Induction

① Period T :

$$\max_{B_{T+1} \geq 0} \log(B_T - B_{T+1})$$

$$\Rightarrow B_{T+1} = 0, \quad C_T = B_T, \quad V_T = \log(B_T)$$

② Period $T - 1$:

$$\max_{B_T} \left\{ \log(B_{T-1} - B_T) + \beta V_T(B_T) \right\}$$

$$\Rightarrow B_T = \frac{\beta}{1 + \beta} B_{T-1}, \quad C_{T-1} = \frac{1}{1 + \beta} B_{T-1},$$

$$V_{T-1} = (1 + \beta) \log \frac{B_{T-1}}{1 + \beta} + \beta \log(\beta)$$

Backward Induction

③ Period $T - 2$:

$$\max_{B_{T-1}} \left\{ \log(B_{T-2} - B_{T-1}) + \beta V_{T-1}(B_{T-1}) \right\}$$

Backward Induction

③ Period $T - 2$:

$$\max_{B_{T-1}} \left\{ \log(B_{T-2} - B_{T-1}) + \beta V_{T-1}(B_{T-1}) \right\}$$

$$\Rightarrow B_{T-1} = \frac{\beta(1 + \beta)}{1 + \beta(1 + \beta)} B_{T-2}, \quad C_{T-2} = \frac{1}{1 + \beta(1 + \beta)} B_{T-2},$$

$$V_{T-2} = (1 + \beta + \beta^2) \log \frac{B_{T-2}}{1 + \beta + \beta^2} + (\beta + 2\beta^2) \log(\beta)$$

Backward Induction

③ Period $T - 2$:

$$\max_{B_{T-1}} \left\{ \log(B_{T-2} - B_{T-1}) + \beta V_{T-1}(B_{T-1}) \right\}$$

$$\Rightarrow B_{T-1} = \frac{\beta(1+\beta)}{1+\beta(1+\beta)} B_{T-2}, \quad C_{T-2} = \frac{1}{1+\beta(1+\beta)} B_{T-2},$$

$$V_{T-2} = (1 + \beta + \beta^2) \log \frac{B_{T-2}}{1 + \beta + \beta^2} + (\beta + 2\beta^2) \log(\beta)$$

④ ...

Backward Induction

③ Period $T - 2$:

$$\max_{B_{T-1}} \left\{ \log(B_{T-2} - B_{T-1}) + \beta V_{T-1}(B_{T-1}) \right\}$$

$$\Rightarrow B_{T-1} = \frac{\beta(1 + \beta)}{1 + \beta(1 + \beta)} B_{T-2}, \quad C_{T-2} = \frac{1}{1 + \beta(1 + \beta)} B_{T-2},$$

$$V_{T-2} = (1 + \beta + \beta^2) \log \frac{B_{T-2}}{1 + \beta + \beta^2} + (\beta + 2\beta^2) \log(\beta)$$

④ ...

⑤ Period t : conjecture $V_t = (1 + \beta + \dots + \beta^{T-t}) \log B_t + \alpha_t$

Backward Induction

③ Period $T - 2$:

$$\max_{B_{T-1}} \left\{ \log(B_{T-2} - B_{T-1}) + \beta V_{T-1}(B_{T-1}) \right\}$$

$$\Rightarrow B_{T-1} = \frac{\beta(1 + \beta)}{1 + \beta(1 + \beta)} B_{T-2}, \quad C_{T-2} = \frac{1}{1 + \beta(1 + \beta)} B_{T-2},$$

$$V_{T-2} = (1 + \beta + \beta^2) \log \frac{B_{T-2}}{1 + \beta + \beta^2} + (\beta + 2\beta^2) \log(\beta)$$

④ ...

⑤ Period t : conjecture $V_t = (1 + \beta + \dots + \beta^{T-t}) \log B_t + \alpha_t$

$$\max_{B_{t+1}} \left\{ \log(B_t - B_{t+1}) + \beta(1 + \dots + \beta^{T-t-1}) \log B_{t+1} + \beta \alpha_{t+1} \right\}$$

Backward Induction

③ Period $T - 2$:

$$\max_{B_{T-1}} \left\{ \log(B_{T-2} - B_{T-1}) + \beta V_{T-1}(B_{T-1}) \right\}$$

$$\Rightarrow B_{T-1} = \frac{\beta(1+\beta)}{1+\beta(1+\beta)} B_{T-2}, \quad C_{T-2} = \frac{1}{1+\beta(1+\beta)} B_{T-2},$$

$$V_{T-2} = (1 + \beta + \beta^2) \log \frac{B_{T-2}}{1 + \beta + \beta^2} + (\beta + 2\beta^2) \log(\beta)$$

④ ...

⑤ Period t : conjecture $V_t = (1 + \beta + \dots + \beta^{T-t}) \log B_t + \alpha_t$

$$\max_{B_{t+1}} \left\{ \log(B_t - B_{t+1}) + \beta(1 + \dots + \beta^{T-t-1}) \log B_{t+1} + \beta \alpha_{t+1} \right\}$$

$$\Rightarrow B_{t+1} = \frac{\beta + \dots + \beta^{T-t}}{1 + \dots + \beta^{T-t}} B_t, \quad C_t = \frac{1}{1 + \dots + \beta^{T-t}} B_t,$$

$$V_t = (1 + \dots + \beta^{T-t}) \log B_t + \alpha_t$$

Infinite Horizon

- Surprisingly, infinite horizon simplifies the problem!

$$V_t(B) = \max_{B'} \left\{ \log(B - B') + \beta V_{t+1}(B') \right\}$$

Infinite Horizon

- Surprisingly, infinite horizon simplifies the problem!

$$V_t(B) = \max_{B'} \left\{ \log(B - B') + \beta V_{t+1}(B') \right\}$$

- Conjecture $V(B) = \frac{1}{1-\beta} \log B + \alpha$ and solve

$$\max_{B'} \left\{ \log(B - B') + \frac{\beta}{1-\beta} \log B' + \beta\alpha \right\}$$

Infinite Horizon

- Surprisingly, infinite horizon simplifies the problem!

$$V_t(B) = \max_{B'} \left\{ \log(B - B') + \beta V_{t+1}(B') \right\}$$

- Conjecture $V(B) = \frac{1}{1-\beta} \log B + \alpha$ and solve

$$\max_{B'} \left\{ \log(B - B') + \frac{\beta}{1-\beta} \log B' + \beta\alpha \right\}$$

- Optimal choice:

$$\begin{aligned} B_{t+1} &= \beta B_t, & C_t &= (1 - \beta) B_t \\ V_t &= \frac{1}{1 - \beta} \log B_t + \log(1 - \beta) + \frac{\beta}{1 - \beta} \log \beta + \beta\alpha \end{aligned}$$

Infinite Horizon

- Surprisingly, infinite horizon simplifies the problem!

$$V_t(B) = \max_{B'} \left\{ \log(B - B') + \beta V_{t+1}(B') \right\}$$

- Conjecture $V(B) = \frac{1}{1-\beta} \log B + \alpha$ and solve

$$\max_{B'} \left\{ \log(B - B') + \frac{\beta}{1-\beta} \log B' + \beta\alpha \right\}$$

- Optimal choice:

$$\begin{aligned} B_{t+1} &= \beta B_t, & C_t &= (1 - \beta) B_t \\ V_t &= \frac{1}{1-\beta} \log B_t + \log(1 - \beta) + \frac{\beta}{1-\beta} \log \beta + \beta\alpha \end{aligned}$$

- Solve for the constant term:

$$\alpha = \frac{1}{1-\beta} \log(1 - \beta) + \frac{\beta}{(1-\beta)^2} \log \beta$$

THEORY

Bellman Equation

- Consider a generalized **Bellman equation**:

$$V(B) = \max_{B' \in \Gamma(B)} \left\{ u(B, B') + \beta V(B') \right\}$$

where $\Gamma(\cdot)$ is a non-empty correspondence

- state variable B summarizes the past
- **value function** $V(\cdot)$ summarizes the future
- **policy function** $B' = g(B) \Rightarrow B_1 = g(B_0), B_2 = g(B_1) \dots$
- **functional equation** with unknown $V(\cdot)$

Principle of Optimality

- **Theorem:** sequential problem is equivalent to the Bellman equation

Principle of Optimality

- **Theorem:** sequential problem is equivalent to the Bellman equation
- Define the value function:

$$V(B_t) = \max_{\{B_{t+j+1} \in \Gamma(B_{t+j})\}_{j=0}^{\infty}} \sum_{j=0}^{\infty} \beta^j u(B_{t+j}, B_{t+j+1})$$

Principle of Optimality

- **Theorem:** sequential problem is equivalent to the Bellman equation
- Define the value function:

$$V(B_t) = \max_{\{B_{t+j+1} \in \Gamma(B_{t+j})\}_{j=0}^{\infty}} \sum_{j=0}^{\infty} \beta^j u(B_{t+j}, B_{t+j+1})$$

- Sketch of proof ($\Gamma(\cdot)$ is suppressed for simplicity):

$$\begin{aligned} V(B_t) &= \max_{\{B_{t+j+1}\}_{j=0}^{\infty}} \left\{ u(B_t, B_{t+1}) + \sum_{j=1}^{\infty} \beta^j u(B_{t+j}, B_{t+j+1}) \right\} \\ &= \max_{B_{t+1}} \left\{ u(B_t, B_{t+1}) + \max_{\{B_{t+j+1}\}_{j=1}^{\infty}} \sum_{j=1}^{\infty} \beta^j u(B_{t+j}, B_{t+j+1}) \right\} \\ &= \max_{B_{t+1}} \left\{ u(B_t, B_{t+1}) + \max_{\{B_{(t+1)+j+1}\}_{j=0}^{\infty}} \sum_{j=0}^{\infty} \beta^{j+1} u(B_{(t+1)+j}, B_{(t+1)+j+1}) \right\} \\ &= \max_{B_{t+1}} \left\{ u(B_t, B_{t+1}) + \beta V(B_{t+1}) \right\} \end{aligned}$$

APPLICATIONS

Optimality Conditions

- Bellman equation can be used to obtain the EE

$$V(B) = \max_{B'} \left\{ u(RB - B') + \beta V(B') \right\}$$

Optimality Conditions

- Bellman equation can be used to obtain the EE

$$V(B) = \max_{B'} \left\{ u(RB - B') + \beta V(B') \right\}$$

- First-order condition:

$$u'(RB - B') = \beta V'(B')$$

- Envelope condition

$$V'(B) = Ru'(RB - B')$$

Optimality Conditions

- Bellman equation can be used to obtain the EE

$$V(B) = \max_{B'} \left\{ u(RB - B') + \beta V(B') \right\}$$

- First-order condition:

$$u'(RB - B') = \beta V'(B')$$

- Envelope condition

$$V'(B) = Ru'(RB - B')$$

- Combine the two equation:

$$u'(C) = \beta Ru'(C')$$

Numerical Algorithm

- How to solve the BE when no analytical $V(\cdot)$ exists?

Numerical Algorithm

- How to solve the BE when no analytical $V(\cdot)$ exists?
- The idea that $V_T(\cdot) \xrightarrow{T} V(\cdot)$ suggests the following approach:
 - 1 start with some initial guess for $V_0(\cdot)$
 - 2 solve the Bellman equation to obtain $V_1(\cdot)$

$$V_1(B) = \max_{B'} \left\{ u(B, B') + \beta V_0(B') \right\}$$

- 3 iterate and update the value function at each step

$$V_{j+1}(B) = \max_{B'} \left\{ u(B, B') + \beta V_j(B') \right\}$$

- 4 stop when the value function converges
- 5 solve for the policy function $B' = g(B)$

CONTINUOUS TIME

HJB equation

- Consider deterministic savings problem
- Bellman equation for period of arbitrary length Δ :

$$V(B_t) = \max_{C_t} \left\{ u(C_t)\Delta + e^{-\rho\Delta} V(B_{t+\Delta}) \right\}$$

HJB equation

- Consider deterministic savings problem
- Bellman equation for period of arbitrary length Δ :

$$V(B_t) = \max_{C_t} \left\{ u(C_t)\Delta + e^{-\rho\Delta} V(B_{t+\Delta}) \right\}$$

- Subtract $V(B_t)$ and use approximation $e^{-\rho\Delta} \approx 1 - \rho\Delta$:

$$\max_{C_t} \left\{ u(C_t)\Delta + (1 - \rho\Delta)V(B_{t+\Delta}) - V(B_t) \right\} = 0$$

HJB equation

- Consider deterministic savings problem

- Bellman equation for period of arbitrary length Δ :

$$V(B_t) = \max_{C_t} \left\{ u(C_t)\Delta + e^{-\rho\Delta} V(B_{t+\Delta}) \right\}$$

- Subtract $V(B_t)$ and use approximation $e^{-\rho\Delta} \approx 1 - \rho\Delta$:

$$\max_{C_t} \left\{ u(C_t)\Delta + (1 - \rho\Delta)V(B_{t+\Delta}) - V(B_t) \right\} = 0$$

- Divide by Δ and take the limit $\Delta \rightarrow 0$:

$$\max_{C_t} \left\{ u(C_t) - \rho V(B_t) + V'(B_t)\dot{B}_t \right\} = 0$$

HJB equation

- Consider deterministic savings problem

- Bellman equation for period of arbitrary length Δ :

$$V(B_t) = \max_{C_t} \left\{ u(C_t)\Delta + e^{-\rho\Delta} V(B_{t+\Delta}) \right\}$$

- Subtract $V(B_t)$ and use approximation $e^{-\rho\Delta} \approx 1 - \rho\Delta$:

$$\max_{C_t} \left\{ u(C_t)\Delta + (1 - \rho\Delta)V(B_{t+\Delta}) - V(B_t) \right\} = 0$$

- Divide by Δ and take the limit $\Delta \rightarrow 0$:

$$\max_{C_t} \left\{ u(C_t) - \rho V(B_t) + V'(B_t)\dot{B}_t \right\} = 0$$

- Hamilton-Jacobi-Bellman (HJB) equation:**

$$\rho V(B) = \max_C \left\{ u(C) + V'(B)(rB - C) \right\}$$

Euler Equation

- HJB equation can be used to derive the optimality conditions

Euler Equation

- HJB equation can be used to derive the optimality conditions
- First-order condition:

$$u'(C_t) = V'(B_t)$$

- Envelope condition:

$$\rho V'(B_t) = rV'(B_t) + V''(B_t)\dot{B}_t$$

Euler Equation

- HJB equation can be used to derive the optimality conditions
- First-order condition:

$$u'(C_t) = V'(B_t)$$

- Envelope condition:

$$\rho V'(B_t) = rV'(B_t) + V''(B_t)\dot{B}_t$$

- Differentiate FOC wrt t :

$$u''(C_t)\dot{C}_t = V''(B_t)\dot{B}_t$$

- Combine together to get the Euler equation:

$$-\frac{u''(C_t)}{u'(C_t)}\dot{C}_t = r - \rho$$

Example

- Consider again the case of $u(C) = \log C$
- HJB equation:

$$\rho V(B) = \max_C \left\{ u(C) + V'(B)(rB - C) \right\}$$

- Conjecture that $V(B) = \alpha + \gamma \log B$

$$\text{FOC: } \frac{1}{C} = \frac{\gamma}{B} \quad \Rightarrow \quad C = \frac{B}{\gamma}$$

- Substitute into the HJB equation:

$$\rho \left[\alpha + \gamma \log B \right] = \log \frac{B}{\gamma} + \frac{\gamma}{B} \left(rB - \frac{B}{\gamma} \right)$$

- Collecting terms, we get

$$\gamma = \frac{1}{\rho}, \quad \alpha = \frac{\log \rho + r/\rho - 1}{\rho}$$

REINFORCEMENT LEARNING

Main Idea

- Two limitations of dynamic programming:
 - ① too expensive to solve explicitly (e.g. chess)
 - ② transition/reward structure may be unknown (e.g. robotics)
- **Reinforcement learning** solves both and is a cornerstone of AI
- Consider pie eating problem with discrete $\{B^1 \dots B^N\}$
- Instead of value function define **state-action value function**:

$$Q(B^i, B^j) = u(RB^i - B^j) + \beta V(B^j),$$

$$\text{where } V(B^i) = \max_j Q(B^i, B^j)$$

Algorithm

- 1 Start with an arbitrary guess for $Q_0(\cdot, \cdot)$
- 2 For each visited state B^i , choose the next action B^j using the current estimate of the state-action function:

$$B^j = \begin{cases} \arg \max_k Q_{n+1}(B^i, B^k), & \text{w/p } 1 - \varepsilon \\ \text{a random feasible action,} & \text{w/p } \varepsilon \end{cases}$$

- 3 Update after visiting a new state-action pair (B^i, B^j) :

$$Q_{n+1}(B^i, B^j) = u(RB^i - B^j) + \beta \max_k Q_n(B^j, B^k)$$

- 4 Update the corresponding value function:

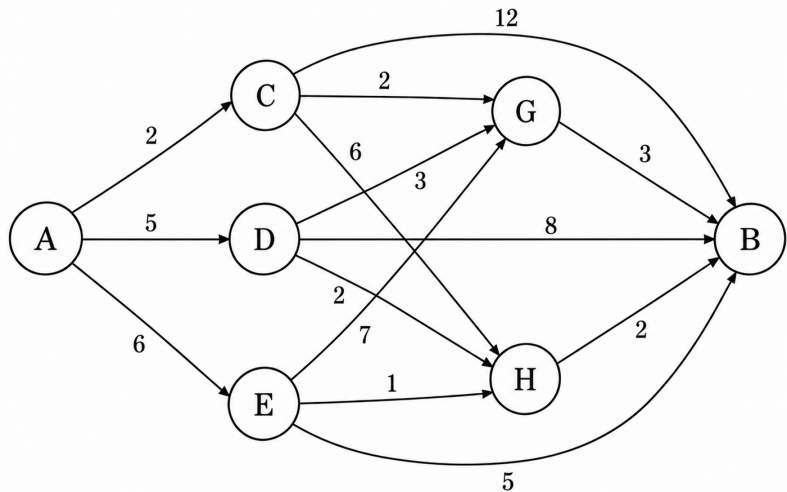
$$V_{n+1}(B^i) = \max_k Q_{n+1}(B^i, B^k)$$

- 5 Continue until the state-action function $Q(\cdot, \cdot)$ converges and then recover the optimal policy function from

$$g(B^i) = \arg \max_j Q(B^i, B^j)$$

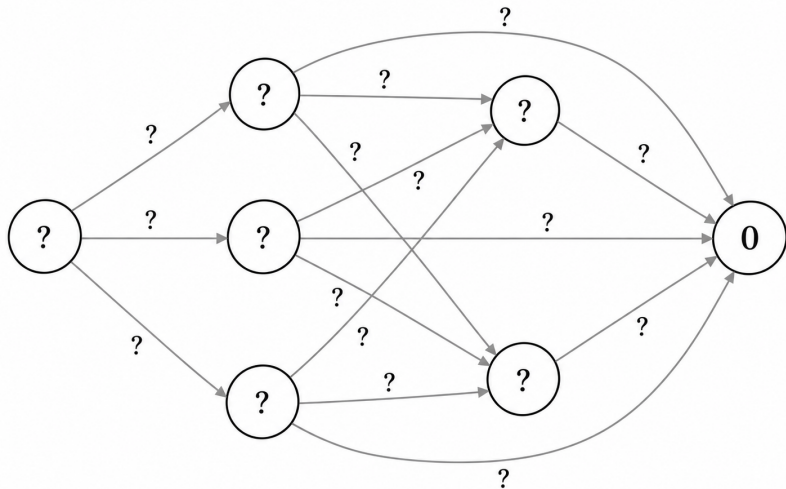
Back to Transportation Problem

Transportation network with multiple possible routes



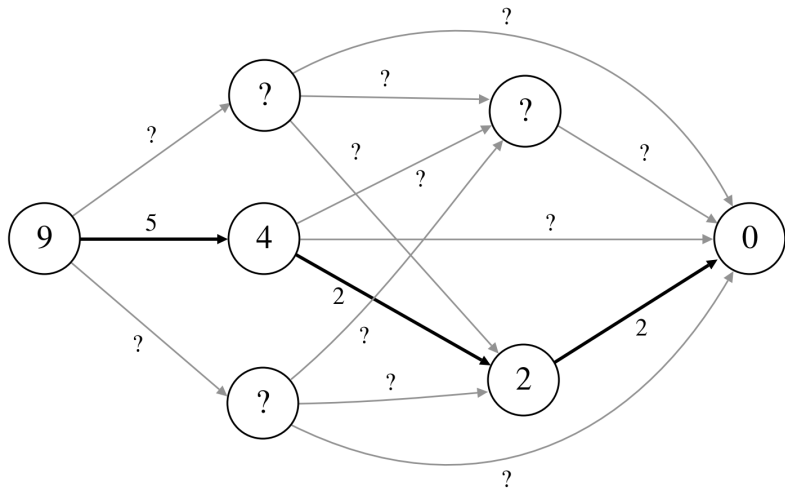
Back to Transportation Problem

RL: initial knowledge



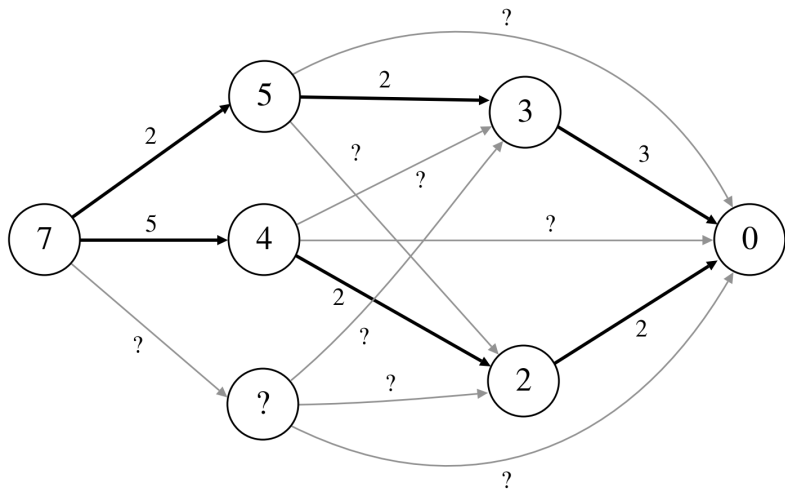
Back to Transportation Problem

RL: after 1 trip



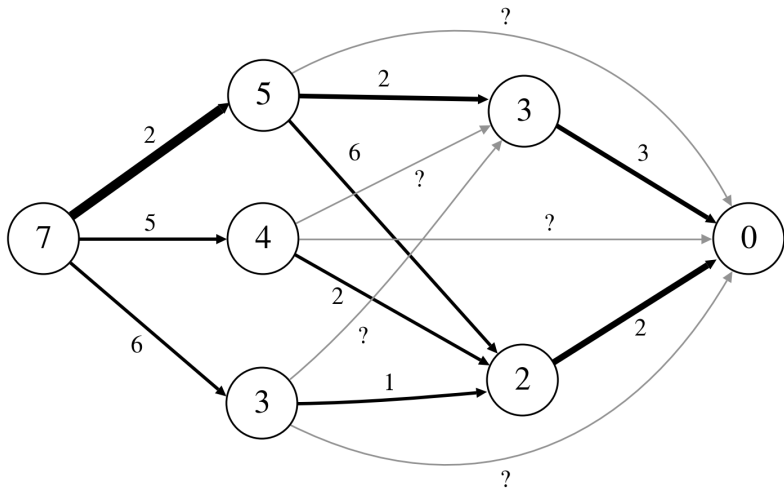
Back to Transportation Problem

RL: after 2 trips



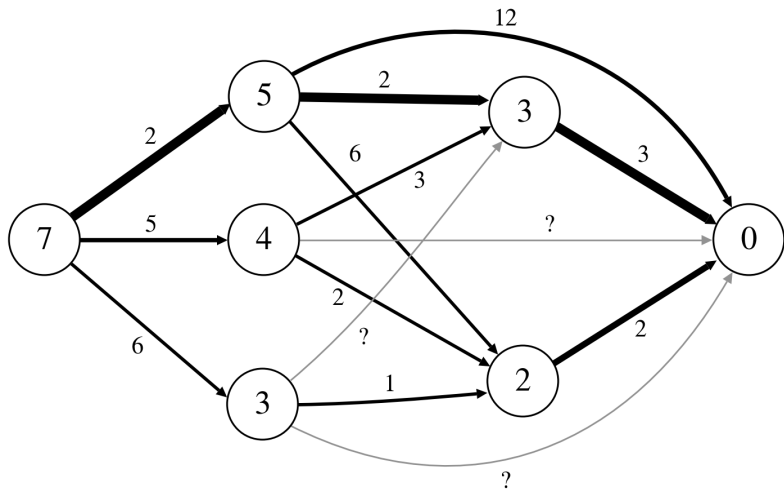
Back to Transportation Problem

RL: after 5 trips



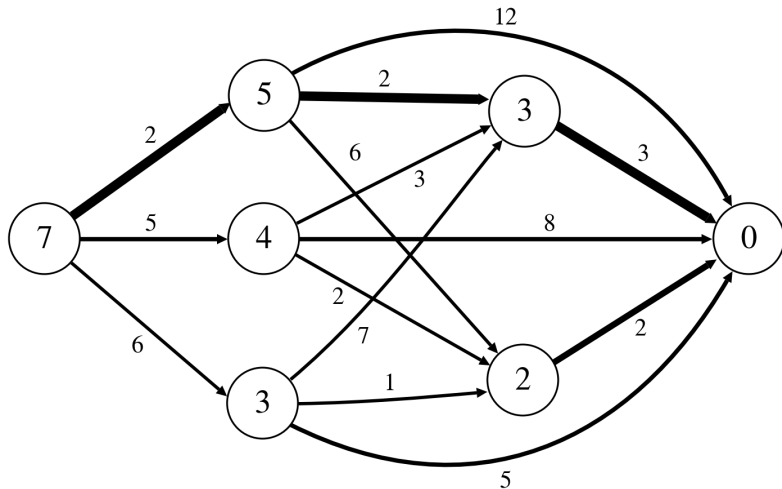
Back to Transportation Problem

RL: after 20 trips



Back to Transportation Problem

RL: after 100 trips



Summary

- Optimization methods:

time \ approach	sequential	recursive
discrete	Lagrangian	Bellman equation
continuous	Hamiltonian	HJB equation

Summary

- Optimization methods:

time \ approach	sequential	recursive
discrete	Lagrangian	Bellman equation
continuous	Hamiltonian	HJB equation

- Solution methods:

local	global
linearization + Blanchard-Kahn	value function iteration

Summary

- Optimization methods:

time \ approach	sequential	recursive
discrete	Lagrangian	Bellman equation
continuous	Hamiltonian	HJB equation

- Solution methods:

local	global
linearization + Blanchard-Kahn	value function iteration

- Simple extensions for stochastic models